

STAT 432: Homework 02

Jacob Jacobson

September 2026

Question 1: Training and test error under a fixed design

```
set.seed(43202)
n <- 100; p <- 20; repetitions <- 200
X_all <- matrix(rnorm(n * p), nrow = n)
beta <- 0.4^sqrt(1:p)
mu <- drop(X_all %*% beta)
train_mse <- test_mse <- matrix(NA_real_, repetitions, p + 1)

for (k in seq_len(repetitions)) {
  y <- mu + rnorm(n)
  y_star <- mu + rnorm(n)
  for (m in 0:p) {
    X_m <- cbind(1, X_all[, seq_len(m), drop = FALSE])
    Q <- qr.Q(qr(X_m))
    y_hat <- drop(Q %*% crossprod(Q, y))
    train_mse[k, m + 1] <- mean((y - y_hat)^2)
    test_mse[k, m + 1] <- mean((y_star - y_hat)^2)
  }
}

theory_train <- theory_test <- bias_mse <- numeric(p + 1)
for (m in 0:p) {
  X_m <- cbind(1, X_all[, seq_len(m), drop = FALSE])
  Q <- qr.Q(qr(X_m))
  mu_hat <- drop(Q %*% crossprod(Q, mu))
  bias_mse[m + 1] <- mean((mu - mu_hat)^2)
  theory_train[m + 1] <- bias_mse[m + 1] + 1 - (m + 1) / n
  theory_test[m + 1] <- bias_mse[m + 1] + 1 + (m + 1) / n
}

average_train <- colMeans(train_mse)
average_test <- colMeans(test_mse)
monotone_runs <- apply(train_mse, 1, function(z) all(diff(z) <= 1e-10))
cat("Runs with nonincreasing training MSE:", sum(monotone_runs), "of", repetitions, "\n")

## Runs with nonincreasing training MSE: 200 of 200

cat("Largest adjacent change:", max(apply(train_mse, 1, function(z) diff(z))), "\n")

## Largest adjacent change: -4.1287e-11
```

```

cat("Minimum average test at m =", which.min(average_test) - 1, "\n")

## Minimum average test at m = 6

cat("Minimum expected test at m =", which.min(theory_test) - 1, "\n")

## Minimum expected test at m = 6

selected <- c(0, 5, 10, 15, 20) + 1
round(data.frame(
  m = 0:p, AverageTrain = average_train, ExpectedTrain = theory_train,
  AverageTest = average_test, ExpectedTest = theory_test
)[selected, ], 4)

##      m AverageTrain ExpectedTrain AverageTest ExpectedTest
## 1    0      1.3147      1.2974      1.3367      1.3174
## 6    5      0.9806      0.9750      1.0979      1.0950
## 11  10      0.9061      0.8992      1.1186      1.1192
## 16  15      0.8524      0.8419      1.1594      1.1619
## 21  20      0.7948      0.7900      1.2170      1.2100

matplot(
  0:p,
  cbind(average_train, theory_train, average_test, theory_test, test_mse[1, ]),
  type = "l", lty = c(1, 2, 1, 2, 3), lwd = c(2.4, 2, 2.4, 2, 1.5),
  col = c("#2166AC", "#2166AC", "#B2182B", "#B2182B", "gray45"),
  xlab = "Number of predictors (m)", ylab = "Mean squared error", bty = "l"
)
legend("topright", bty = "n", cex = .78, lwd = 2,
  legend = c("Average training", "Expected training", "Average test",
    "Expected test", "Test, run 1"),
  col = c("#2166AC", "#2166AC", "#B2182B", "#B2182B", "gray45"),
  lty = c(1, 2, 1, 2, 3))

```

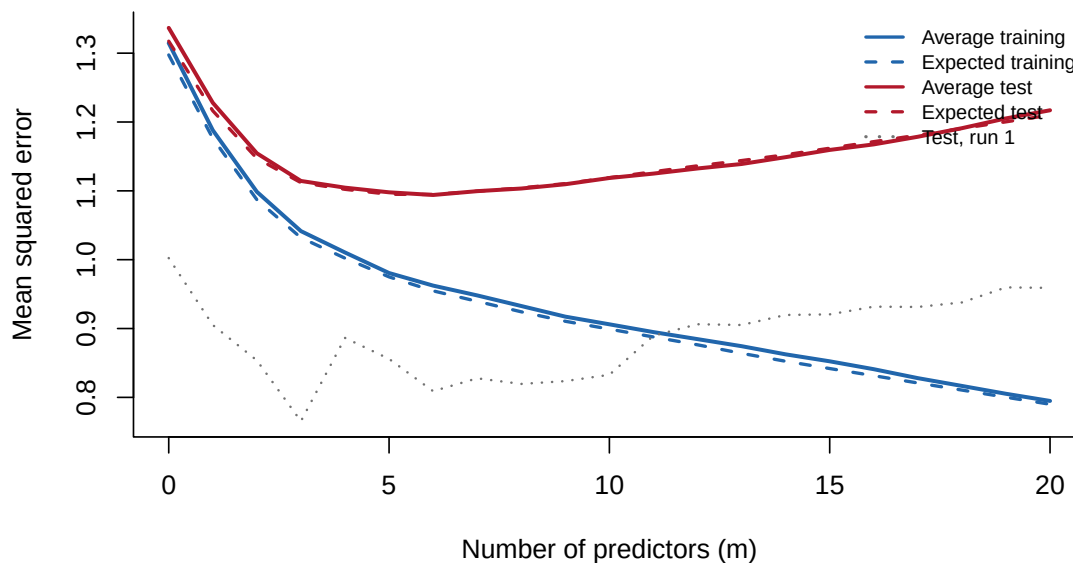


Figure 1: Training and test MSE along the nested model sequence.

All 200 training curves are nonincreasing (up to numerical tolerance). This must happen because the fitted-value space for model m is contained in that for model $m + 1$: least squares can reuse the old coefficients, so its minimized RSS cannot be larger.

The Monte Carlo averages closely track the conditional expectations. A single test curve is much rougher because it retains the randomness in both the fitted training response and its particular independent test response; averaging 200 runs cancels much of this noise.

Expected test MSE is

$$B_m^2/n + 1 + (m + 1)/n.$$

Adding predictors generally reduces approximation bias B_m^2/n , but increases estimation variance by $1/n$ per fitted coefficient. Expected training MSE has the opposite complexity term, $-(m + 1)/n$, because additional flexibility fits training noise. Therefore training error alone continually rewards complexity and cannot honestly select the predictor count.

Question 2: Prediction error at one target point

Here $\mu_0 = \mathbf{x}_0^T \boldsymbol{\beta} = 0.5 + 0.5 = 1$. The omitted predictors contribute a deterministic target-point bias, while estimating the intercept and included slopes contributes variance. Orthogonality makes these coefficient-estimation variances add.

m	Squared bias	Estimation variance	Expected squared error
0	1.00	0.01	1.01
1	1.00	0.01	1.01
2	0.25	0.02	0.27
3	0.25	0.02	0.27
4	0.25	0.02	0.27
5	0.00	0.03	0.03
6	0.00	0.03	0.03

The theoretical error changes when predictor 2 or 5 enters. It remains unchanged from $m = 0$ to 1, from $m = 2$ through 4, and from $m = 5$ to 6.

```
set.seed(43203)
n <- 100; p <- 6; repetitions <- 200; sigma2 <- 1
X_all <- outer(1:n, 1:p,
  function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n))
beta <- rep(0.5, p)
x0 <- c(0, 1, 0, 0, 1, 0)
mu <- drop(X_all %*% beta); mu0 <- sum(x0 * beta)
simulation_error <- matrix(NA_real_, repetitions, p + 1)

for (k in seq_len(repetitions)) {
  y <- mu + rnorm(n)
  for (m in 0:p) {
    X_m <- cbind(1, X_all[, seq_len(m), drop = FALSE])
    bhat <- qr.solve(X_m, y)
    mu0_hat <- sum(c(1, x0[seq_len(m)]) * bhat)
    simulation_error[k, m + 1] <- (mu0_hat - mu0)^2
  }
}

theoretical <- numeric(p + 1)
for (m in 0:p) {
  omitted <- if (m < p) (m + 1):p else integer(0)
  squared_bias <- sum(x0[omitted] * beta[omitted])^2
  variance <- sigma2 * (1 + sum(x0[seq_len(m)]^2)) / n
  theoretical[m + 1] <- squared_bias + variance
}
simulated <- colMeans(simulation_error)
round(data.frame(m = 0:p, Simulated = simulated, Theoretical = theoretical), 4)

##   m Simulated Theoretical
## 1 0    1.0189    1.01
## 2 1    1.0189    1.01
## 3 2    0.2671    0.27
## 4 3    0.2671    0.27
## 5 4    0.2671    0.27
## 6 5    0.0338    0.03
## 7 6    0.0338    0.03

cat("Smallest theoretical minimizer: m =", which.min(theoretical) - 1, "\n")

## Smallest theoretical minimizer: m = 5
```

```

plot(0:p, simulated, type = "o", pch = 16, lwd = 2.2, col = "#2166AC",
     xlab = "Number of predictors (m)", ylab = "Squared error at target", bty = "l")
lines(0:p, theoretical, lty = 2, lwd = 2.2, col = "#B2182B")
legend("topright", bty = "n", lwd = 2, pch = c(16, NA), lty = c(1, 2),
     col = c("#2166AC", "#B2182B"),
     legend = c("Simulation average", "Theoretical expectation"))

```

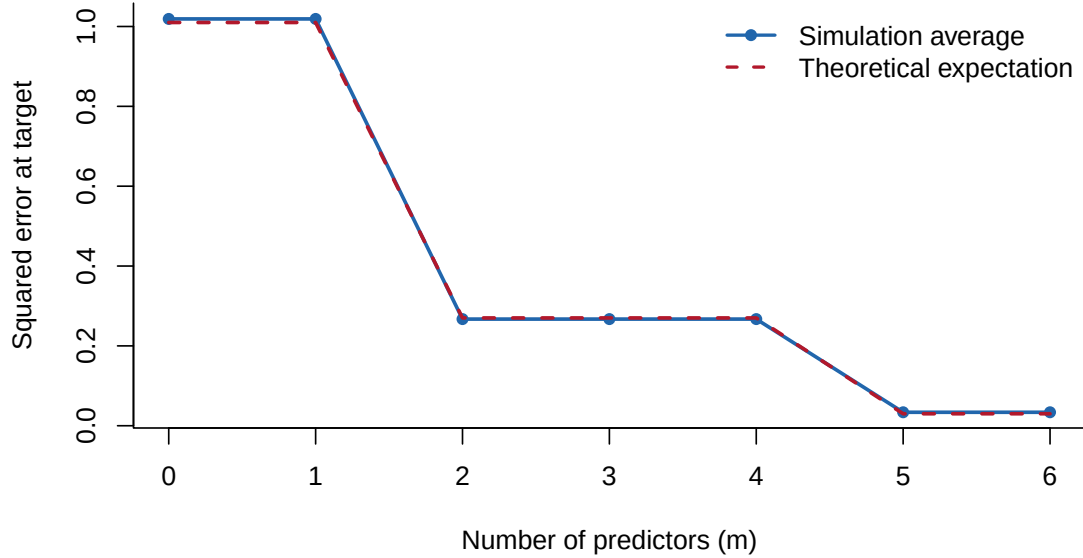


Figure 2: Simulation and theoretical squared error at the target point.

The smallest theoretical minimizer is $m = 5$. Although every coefficient is nonzero, only predictors 2 and 5 directly affect this target because only x_{02} and x_{05} are nonzero. Average test MSE over a design weights a predictor wherever it varies across all design rows, so a predictor irrelevant at this one target can still matter for average prediction.

Question 3: The optimism correction

Let $A = I - H$. The training residual is

$$y - Hy = A\mu + A\epsilon.$$

The cross term has expectation zero. Since A is symmetric and idempotent,

$$\begin{aligned}
 E\{\|y - Hy\|^2 \mid X\} &= \|A\mu\|^2 + E(\epsilon^T A\epsilon \mid X) \\
 &= B^2 + \sigma^2 \text{tr}(I - H) \\
 &= B^2 + \sigma^2\{n - (p + 1)\}.
 \end{aligned}$$

Dividing by n gives

$$E(\text{MSE}_{\text{train}} \mid X) = B^2/n + \sigma^2\{1 - (p + 1)/n\}.$$

For an independent test response,

$$y^* - Hy = (I - H)\mu + \epsilon^* - H\epsilon.$$

The cross terms have expectation zero, and independence plus $H^2 = H$ gives

$$\begin{aligned} E\{\|\epsilon^* - H\epsilon\|^2 \mid X\} &= n\sigma^2 + \sigma^2 \text{tr}(H) \\ &= \sigma^2\{n + (p + 1)\}. \end{aligned}$$

Thus

$$E(\text{MSE}_{\text{test}} \mid X) = B^2/n + \sigma^2\{1 + (p + 1)/n\}.$$

Subtracting yields expected optimism $2(p + 1)\sigma^2/n$. With $n = 120$, $p = 4$, $B^2/n = 0.04$, and $\sigma^2 = 1$,

$$E(\text{MSE}_{\text{train}}) = 0.998333, \quad E(\text{MSE}_{\text{test}}) = 1.081667, \quad \text{difference} = 0.083333.$$

These are expectations, not a pointwise ordering; response noise can make a particular test MSE smaller than its paired training MSE.

For $\text{RSS} = 116.4$ and $\hat{\sigma}^2 = 0.96$,

$$\widehat{\text{MSE}}_{\text{test}} = \frac{116.4 + 2(5)(0.96)}{120} = 1.05, \quad C_p = \frac{116.4}{0.96} - 120 + 10 = 11.25.$$

They satisfy

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\hat{\sigma}^2(C_p + n)}{n}.$$

For common n and $\hat{\sigma}^2$, this is an increasing transformation, so minimizing either quantity gives the same ranking.

Question 4: Comparing Mallows' C_p , AIC, and BIC

```
data_file <- Sys.getenv("DIABETES_CSV", unset = "data/diabetes.csv")
training <- read.csv(data_file, check.names = FALSE)[1:370, ]
n <- nrow(training)
vars <- list(
  A = c("bmi", "bp", "s5", "sex", "s1", "s2", "s4"),
  B = c("bmi", "bp", "s5", "sex", "s1", "s2", "s4", "s6"),
  Full = c("age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6")
)
fit_rss <- function(v) {
  X <- cbind(1, as.matrix(training[v]))
  sum(lm.fit(X, training$y)$residuals^2)
}
rss <- vapply(vars, fit_rss, numeric(1))
p_count <- vapply(vars, length, numeric(1))
residual_df <- n - (p_count["Full"] + 1)
sigma2_hat <- rss["Full"] / residual_df
answer <- data.frame(Model = c("A", "B"), p = p_count[c("A", "B")],
  RSS = rss[c("A", "B")])
answer$Cp <- answer$RSS / sigma2_hat - n + 2 * (answer$p + 1)
answer$AICred <- n * log(answer$RSS / n) + 2 * (answer$p + 1)
answer$BICred <- n * log(answer$RSS / n) + log(n) * (answer$p + 1)
cat("Full-model RSS:", rss["Full"], "\n")
```

```
## Full-model RSS: 1089452

cat("Full-model residual df:", residual_df, "\n")

## Full-model residual df: 359

cat("Common variance estimate:", sigma2_hat, "\n")

## Common variance estimate: 3034.7

print(transform(answer, RSS = round(RSS, 3), Cp = round(Cp, 3),
  AICred = round(AICred, 3), BICred = round(BICred, 3)), row.names = FALSE)

## Model p      RSS      Cp AICred BICred
##      A 7 1098909 8.116 2974.6 3005.9
##      B 8 1089717 7.087 2973.5 3008.8
```

The full-model residual degrees of freedom are $370 - (10 + 1) = 359$; its RSS is 1,089,451.721, giving $\hat{\sigma}^2 = 3034.684$. The results are:

Model	p	RSS	C_p	reduced AIC	reduced BIC
A	7	1,098,908.566	8.116	2,974.640	3,005.948
B	8	1,089,717.057	7.087	2,973.532	3,008.754

Thus C_p and AIC select Model B, whereas BIC selects Model A. On the AIC/BIC scale, adding **s6** improves the fit term by

$$370 \log(\text{RSS}_A / \text{RSS}_B) = 3.108.$$

This exceeds AIC's one-parameter penalty of 2 but not BIC's penalty $\log(370) = 5.914$. On the C_p scale, the fit improvement is $(\text{RSS}_A - \text{RSS}_B) / \hat{\sigma}^2 = 3.029 > 2$, so C_p also accepts **s6**. Selection only chooses between the specified candidates under assumptions and a criterion; it does not establish that either candidate is the true data-generating model.

Question 5: Validation and final test data

(a) Step 2 is the first misuse: the final test responses are used to compare models. Although those responses do not estimate regression coefficients, they do select m . Each of the eleven test MSEs contains random noise, and taking their minimum preferentially selects a value that was unusually low by chance. Reusing that same minimum as the performance estimate is therefore optimistic.

(b) I would reserve rows 371–442 without examining their responses. Within rows 1–370, split observations into five folds. For each candidate m , fit its coefficients on four folds and evaluate MSE on the held-out fold; repeat so each fold is held out once, including all data-dependent preprocessing within each four-fold training set. Select the m with the smallest mean validation MSE, refit that chosen model on all 370 training rows, and evaluate it exactly once on rows 371–442. That final MSE estimates performance of the entire selection-and-refitting procedure.

(c) If all eleven final-test MSEs have already been viewed, they can be reported transparently as exploratory comparisons, along with the fact that the test set influenced selection. The minimum cannot be described as an untouched final evaluation. A new independent test set or genuinely future observations are required for a new final assessment.